ACIT 3855 – Service Oriented Architectures – Assignment 1

Instructor	Mike Mulder (mmulder10@bcit.ca) – Set A Rafi Mohammad (rafi_mohammad@bcit.ca) – Sets B and C
Total Marks	20
Due Dates	End of Last Class: • April 15th for Sets B and C • April 16th for Set A **It can be demoed concurrently with the last lab if necessary.**

This assignment is to be completed and submitted individually.

This assignment requires that you modify your existing services and create a new service. Please treat this as practice for the final exam as you will also have to code a new service with a simple datastore.

Updates to Existing Backend Services

Add a health endpoint to each of your existing backend services (Receiver, Storage, Processing and Analyzer).

GET /health

This endpoint should return 200 always (unless of course if the service is not running).
Make sure to update both the OpenAPI specification and the code for each service.

New Service

Create a new Health Check service running on Port 8120. It should poll the GET /health endpoint of each of the existing services at a reasonable interval (make sure you can justify your interval choice). If it gets a 200 response within a reasonable timeout it should mark that service as up (and running), otherwise it should mark the service as down. Note that if you are using the httpx module, you must change the timeout on your requests to a lower value (i.e., 5 seconds) so you are not waiting too long to determine if the service is healthy.

It must store the status of each of the Receiver, Storage, Processing and Analyzer services and the timestamp at which it last obtained the status in either a JSON based or SQLite based data store.

It must provide an API that returns the status of all the services based on the current statuses in the data store. The JSON response should look something like:

```
{
    "receiver": "Up",
```

```
    "storage": "Down",
    "processing: "Up",
    "analyzer": "Up"
    "last_update": "2026-03-22T11:12:23"
}
```

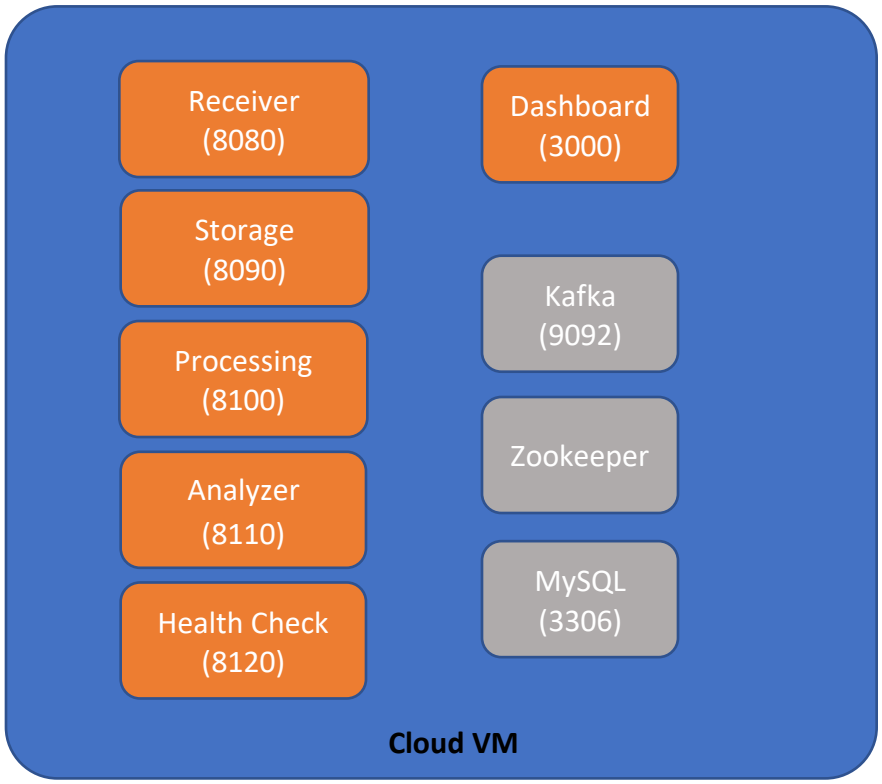
Make sure to include the following in your implementation of the service:

- Configurations
- Logging
- Database creation scripts if using SQLite
- Follows good Python coding conventions (i.e., PEP-8)

For logging, at minimum make sure you log each time you record the status of a service and each time the health status of all the services is retrieved through the API.

Make sure all data files, API endpoints, etc. are maintained in a configuration file.

Deploy your Health Check service as part of your system. This means it must have a dockerfile, requirements.txt and be integrated in your docker-compose.yml file.



Updates to Existing Frontend Dashboard

The Dashboard should show the current statuses of each of the backend services. Like the statistics, the statuses should be updated periodically.

It should reflect the data in the API JSON response, but should be formatted nicely, such as:

Receiver: Up
Storage: Down
Processing: Up
Analyzer: Up
Last Update: 10 seconds ago

<u>Submission</u>

Upload a zip file containing the code for your Health Check service, including:

- app.py
- application and log configuration files
- requirements.txt and dockerfile
- OpenAPI specification
- Database creation script (if applicable)
- Updated docker-compose.yaml file

<u>Evaluation</u>

The grades for each item will be based on the quality of the demo and submission.

Live Demonstration and Q&A • Added /health endpoint to existing services (1 mark) • New Health Check service is running as part of your system as a Docker container that can be started and stopped with Docker Compose (4 marks) • Health Status of each of Receiver, Storage, Processing and Analyzer is correct when services are up or down is available through the API (2 marks) • Health Status of each of Receiver, Storage, Processing and Analyzer is correct when services are up or down is available through the Dashboard UI (2 marks) • Answer at least 3 questions on the design/code and justify your choices (6 marks)	15
Health Check Service Code • Code is correct in terms of the RESTful API and data persistence (2 marks)	5

• Uses OpenAPI, AP Scheduler, JSON OR Sqlite for DB, Docker for deployment (1 marks) • Code uses logging and external configurations (1 marks) • Code Follows Coding Standards (Naming, DocString, Etc) (2 marks) • Code is logical and well-organized (1 marks)	
Total	**20 marks**